



Gelişmiş Mini HBYS (Hastane Bilgi Yönetim Sistemi) Proje Planı

1. Projenin Genel Tanımı

Bu proje, gerçek bir Hastane Bilgi Yönetim Sistemi'nin (HBYS) sadeleştirilmiş ancak profesyonel standartlara uygun bir versiyonunun geliştirilmesini amaçlar. Stajyerin veri modelleme, API tasarımı, güvenlik ve modern web teknolojileri (React, Node.js, PostgreSQL) konusundaki yetkinliklerini artırmayı hedefler.

2. Proje Kapsamı ve Veri Modeli

Sistem; hasta, doktor, randevu ve muayene kayıtlarını yönetmelidir. Veri bütünlüğünü sağlamak için aşağıdaki tablo yapısı temel alınacaktır.



Veri Tablosu Yapısı

Tablo	Alanlar (Fields)	Açıklama
Patients	id, name, surname, nationalId, birthDate, phone, email	Hasta kimlik ve iletişim bilgileri.
Doctors	id, name, branch, email, password	Doktor bilgileri ve giriş yetkileri.
Appointments	id, patientId, doctorId, datetime, status	Randevu zamanı ve durumu (Bekliyor, Tamamlandı, İptal).
MedicalRecords	id, appointmentId, diagnosis, notes	Muayene sonucu teşhis ve doktor notları.

3. Kullanılacak Teknolojiler

Backend

- **Runtime:** Node.js (Express.js)
- **Veritabanı & ORM:** PostgreSQL + Prisma
- **Validasyon:** Zod veya Joi (Gelen verilerin doğrulanması için)
- **Dokümantasyon:** Swagger (OpenAPI) veya Postman Collection

Frontend

- **Framework:** React
 - **Stil:** Tailwind CSS
 - **Bileşen Kütüphanesi:** Shadcn/UI veya MUI (Opsiyonel)
 - **Form Yönetimi:** React Hook Form
-

4. Geliştirme Planı (Adım Adım)

◆ Aşama 1: Ortam Kurulumu & Veritabanı

1. Node.js ve PostgreSQL kurulumu.
2. Prisma şemasının yukarıdaki tablo yapısına göre tasarlanması.
3. Migration işlemlerinin yapılması ve "seed" verisi (test datası) eklenmesi.

◆ Aşama 2: Backend - Temel API Geliştirme

- **Hasta & Doktor API:** CRUD (Ekleme, Listeleme, Güncelleme) işlemlerinin yazılması.
- **Validasyon:** API'lere gelen verilerin (örneğin T.C. Kimlik No'nun 11 hane olması) Zod/Joi ile kontrol edilmesi.

◆ Aşama 3: Randevu Mantığı (Kritik Aşama)

1. **Çakışma Kontrolü:** Aynı doktorun aynı saatte başka bir randevusu olup olmadığının SQL seviyesinde kontrol edilmesi.
2. **Tarih Kontrolü:** Geçmiş bir tarihe randevu alınmasının engellenmesi.
3. **Statü Yönetimi:** Randevuların iptal edilmesi veya tamamlandı olarak işaretlenmesi.

◆ Aşama 4: Frontend Geliştirme

1. **Dashboard:** Genel bir bakış ve hızlı erişim butonları.
2. **Hasta/Doktor Yönetimi:** Liste ve ekleme formları (React Hook Form ile).

3. **Randevu Takvimi:** Tarihe göre randevuların listelenmesi ve yeni kayıt oluşturma.

♦ Aşama 5: Dokümantasyon & Hata Yönetimi

- Tüm API endpoint'lerinin Swagger ile dokümante edilmesi.
 - Global error handler yapısının kurulması (Kullanıcıya anlamlı hata mesajları dönmek için).
-

5. Bonus / İleri Seviye Görevler

Eğer temel özellikler hızla tamamlanırsa, aşağıdaki maddeler stajyerin gelişimini hızlandıracaktır:

- **Soft Delete:** Verilerin veritabanından tamamen silinmesi yerine `isActive: false` yapılarak saklanması.
 - **Middleware:** Randevu çakışma kontrolünün ayrı bir ara yazılım (middleware) olarak tasarlanması.
 - **Dashboard İstatistikleri:** Toplam hasta sayısı, günlük bekleyen randevu sayısı gibi verilerin ana ekranda gösterilmesi.
-

6. Teslim Kriterleri

- **Çalışan Uygulama:** Backend ve Frontend birbiriyle entegre olmalı.
 - **Veri Doğruluğu:** Randevu çakışma kontrolü hatasız çalışmalı.
 - **Dokümantasyon:** Temiz bir README dosyası ve API dokümantasyonu (Swagger/Postman).
 - **UI/UX:** Kullanıcı dostu ve düzenli bir arayüz tasarımı.
-